

DEPARTMENT OF ELECTRICAL ENGINEERING  
IITH Summer Research Fellowship Programme 2026

PROJECT TECHNICAL DOCUMENTATION

# Scientific Calculator Using Arduino UNO

Software-Based Mathematical Function Implementation  
via Runge-Kutta 4<sup>th</sup> Order Numerical Methods

---

|                 |                             |
|-----------------|-----------------------------|
| Microcontroller | ATmega328P-PU (Arduino UNO) |
| Clock Speed     | 16 MHz Crystal Oscillator   |
| Display         | 16×2 LCD (HD44780)          |
| Input           | 5×5 Push-Button Matrix      |
| Language        | Embedded C (AVR-GCC)        |
| Algorithm       | RK4 Numerical Integration   |
| Flash Used      | 12.3 KB / 32 KB (38%)       |
| RAM Used        | 350 B / 2048 B (17%)        |

## Abstract

This report documents the complete design, implementation, and analysis of a **scientific calculator** built on the **ATmega328P microcontroller** (Arduino UNO). The central challenge is the absence of a hardware Floating Point Unit (FPU) on 8-bit AVR microcontrollers, which makes computing transcendental functions ( $\sin x$ ,  $\ln x$ ,  $e^x$ , etc.) expensive and inaccurate via standard libraries.

The solution reformulates each mathematical function as an **Ordinary Differential Equation (ODE)** and solves it using the **Runge-Kutta 4<sup>th</sup> Order (RK4)** method. A comparative benchmark of five numerical methods confirms that RK4 achieves maximum accuracy at the 32-bit precision floor while consuming only **12.3 KB Flash** and **350 bytes SRAM**.

User input is handled through a **5×5 push-button matrix** via row-column scanning. Expressions are parsed with the **Shunting Yard algorithm** and evaluated in Reverse Polish Notation (RPN). A bug in multi-digit expression tokenisation was identified, root-caused, and fixed in `pars.c`, `keypad.c`, and `main.c`.

**Keywords:** Scientific Calculator, ATmega328P, Runge-Kutta, Numerical Methods, Embedded C, Shunting Yard Algorithm, AVR-GCC

# Contents

---

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>1 Project Overview</b>                                | <b>4</b>  |
| 1.1 Objective                                            | 4         |
| 1.2 Supported Functions                                  | 4         |
| 1.3 End-to-End System Workflow                           | 5         |
| <b>2 Hardware Design</b>                                 | <b>6</b>  |
| 2.1 Component List                                       | 6         |
| 2.2 LCD Connections (4-Bit Mode)                         | 6         |
| 2.3 5×5 Keypad Matrix                                    | 6         |
| 2.3.1 Pin Connections                                    | 7         |
| 2.3.2 Key Layout — Mode 1 (Standard)                     | 7         |
| 2.3.3 Key Layout — Mode 2 (Scientific)                   | 7         |
| 2.3.4 Scanning Principle                                 | 7         |
| <b>3 Software Architecture</b>                           | <b>9</b>  |
| 3.1 Modular File Structure                               | 9         |
| 3.2 Expression Parsing: Shunting Yard Algorithm          | 9         |
| 3.2.1 Operator Precedence Table                          | 10        |
| <b>4 Mathematical Engine — RK4 Method</b>                | <b>11</b> |
| 4.1 ODE Formulation of Functions                         | 11        |
| 4.2 RK4 Update Equations                                 | 11        |
| 4.2.1 Second-Order Case: Sine Function                   | 12        |
| 4.3 Non-RK4 Functions                                    | 12        |
| <b>5 Numerical Methods Comparative Analysis</b>          | <b>13</b> |
| 5.1 Methods Evaluated                                    | 13        |
| 5.2 Hardware Performance on ATmega328P                   | 13        |
| 5.3 Accuracy Comparison for $\sin(x)$ , $h \approx 0.06$ | 13        |
| <b>6 Bug Report and Fix</b>                              | <b>15</b> |
| 6.1 Problem Statement                                    | 15        |
| 6.2 Root Cause Analysis                                  | 15        |
| 6.3 Modules Modified and Fixes Applied                   | 15        |
| 6.3.1 <code>pars.c</code> — Tokeniser Fix                | 15        |
| 6.3.2 <code>keypad.c</code> — Debounce and Index Fix     | 16        |

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| 6.3.3    | main.c — Buffer and Display Fix . . . . . | 16        |
| 6.4      | Verified Output After Fix.....            | 16        |
| <b>7</b> | <b>Build and Upload Process .....</b>     | <b>17</b> |
| 7.1      | Toolchain .....                           | 17        |
| 7.2      | Software Prototype (Android / Linux)..... | 17        |
| 7.3      | Hardware Upload via PlatformIO .....      | 17        |
| <b>8</b> | <b>Results and Conclusions.....</b>       | <b>19</b> |
| 8.1      | Final System Specifications .....         | 19        |
| 8.2      | Key Achievements.....                     | 19        |
| 8.3      | Hardware Limitations .....                | 20        |
| 8.4      | Possible Future Extensions.....           | 20        |

# Project Overview

---

## §1.1 Objective

---

The goal is to build a **fully functional scientific calculator** on an **Arduino UNO** (ATmega328P) *without* relying on pre-built math libraries or hardware floating-point units. Every mathematical function is implemented from first principles using **numerical methods** derived from ODE models.

### Why Bare-Metal Arithmetic?

The ATmega328P has *no hardware FPU*. A single float division costs **505 CPU cycles** — more than twice an integer division. Standard C library math functions use large lookup tables that eat into the 32 KB Flash. This project demonstrates that a *unified* RK4-based numerical framework achieves hardware-limited precision in just 12.3 KB Flash and 350 B RAM.

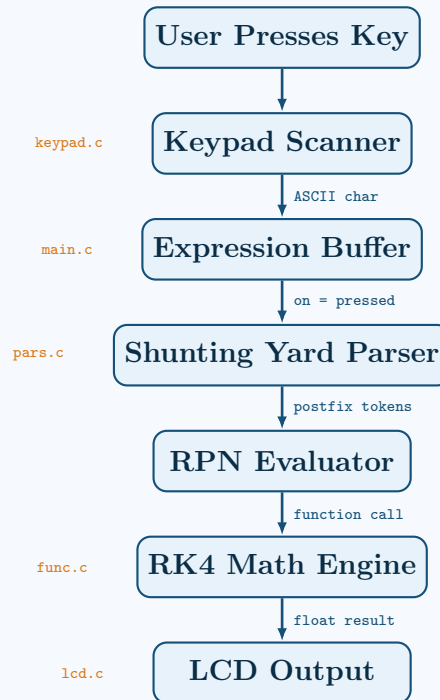
## §1.2 Supported Functions

---

| Category              | Functions / Operations                           |
|-----------------------|--------------------------------------------------|
| Trigonometric         | $\sin(x)$ , $\cos(x)$ , $\tan(x)$                |
| Inverse Trigonometric | $\sin^{-1}(x)$ , $\cos^{-1}(x)$ , $\tan^{-1}(x)$ |
| Exponential / Log     | $e^x$ , $\ln(x)$ , $\log_{10}(x)$                |
| Power / Root          | $x^y$ , $\sqrt{x}$                               |
| Factorial             | $n!$                                             |
| Basic Arithmetic      | $+$ , $-$ , $\times$ , $\div$ , $\text{mod}$     |
| Constants             | $\pi$ , $e$                                      |
| Input                 | Digits 0–9, decimal point, parentheses ( )       |

## §1.3 End-to-End System Workflow

---



# Hardware Design

## §2.1 Component List

| Qty | Component            | Purpose                                        |
|-----|----------------------|------------------------------------------------|
| 1   | Arduino UNO          | Main processing unit; runs calculator firmware |
| 1   | 16×2 LCD (HD44780)   | Displays expression and computed result        |
| 25  | Tactile push-buttons | User input arranged as a 5×5 matrix            |
| 1   | 10 kΩ Potentiometer  | LCD contrast adjustment (VEE pin)              |
| —   | Jumper wires         | Circuit connections                            |

## §2.2 LCD Connections (4-Bit Mode)

| Arduino Pin | LCD Pin | Signal / Description                   |
|-------------|---------|----------------------------------------|
| GND         | VSS (1) | Power ground                           |
| 5V          | VCC (2) | Supply (+5V)                           |
| Pot. wiper  | VEE (3) | Contrast control voltage               |
| D2          | RS (4)  | Register select: 0 = command, 1 = data |
| GND         | R/W (5) | Fixed LOW = Write-only mode            |
| D3          | EN (6)  | Enable strobe                          |
| D4–D7       | DB4–DB7 | 4-bit data bus (pins 11–14)            |
| 5V / GND    | LED±    | Backlight (pins 15–16)                 |

## §2.3 5×5 Keypad Matrix

### 2.3.1 Pin Connections

| Row   | Arduino | Column   | Arduino |
|-------|---------|----------|---------|
| Row 1 | D8      | Column 1 | A0      |
| Row 2 | D9      | Column 2 | A1      |
| Row 3 | D10     | Column 3 | A2      |
| Row 4 | D11     | Column 4 | A3      |
| Row 5 | D12     | Column 5 | A4      |

### 2.3.2 Key Layout — Mode 1 (Standard)

| Col 1     | Col 2     | Col 3     | Col 4    | Col 5     |
|-----------|-----------|-----------|----------|-----------|
| 0         | 1         | 2         | 3        | 4         |
| 5         | 6         | 7         | 8        | 9         |
| +         | -         | $\times$  | $\div$   | $\sin(x)$ |
| $\cos(x)$ | $\tan(x)$ | $\exp(x)$ | $\ln(x)$ | Clear     |
| Backspace | .         | =         | Mode     | $\pi$     |

### 2.3.3 Key Layout — Mode 2 (Scientific)

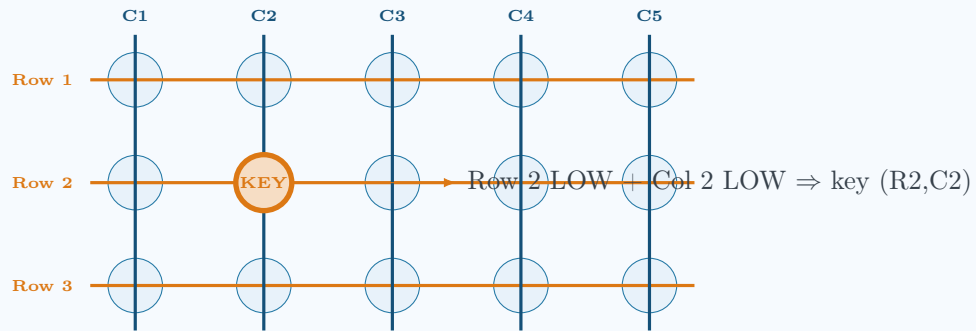
| Col 1          | Col 2          | Col 3          | Col 4           | Col 5          |
|----------------|----------------|----------------|-----------------|----------------|
| 0              | 1              | 2              | 3               | 4              |
| 5              | 6              | 7              | 8               | 9              |
| (              | )              | $x^y$          | <b>fact</b> (x) | $\sin^{-1}(x)$ |
| $\cos^{-1}(x)$ | $\tan^{-1}(x)$ | <b>mod</b> (x) | $\log_{10}(x)$  | Clear          |
| Backspace      | .              | =              | Mode            | $\pi$          |

### 2.3.4 Scanning Principle

The Arduino drives each **row LOW** in turn while reading all **column pins**. A pressed button pulls its column LOW, uniquely identifying the key as:

$$\text{Key} = (\text{Active Row}, \text{Column reading LOW})$$





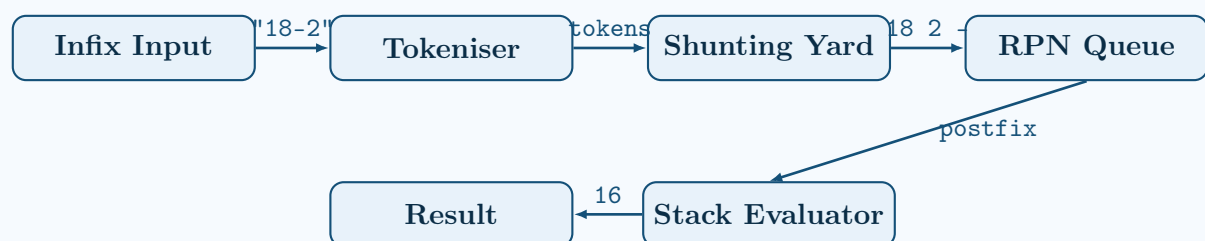
This reduces 25 buttons to just **10 GPIO pins** ( $5 + 5$ ).

# Software Architecture

## §3.1 Modular File Structure

| File                     | Description                                           |
|--------------------------|-------------------------------------------------------|
| <code>defines.h</code>   | Global constants, hardware pin mappings, type aliases |
| <code>main.c</code>      | System init, main event loop, key-press handling      |
| <code>keypad.c/.h</code> | Key scanning, debouncing, ASCII mapping               |
| <code>lcd.c/.h</code>    | LCD driver for HD44780 in 4-bit mode                  |
| <code>pars.c/.h</code>   | Shunting Yard: tokenise, infix→postfix, RPN eval      |
| <code>func.c/.h</code>   | RK4 ODE solver + all math function implementations    |
| <code>gpio.c/.h</code>   | Low-level GPIO pin control wrapper                    |
| <code>timer.c/.h</code>  | Hardware timer configuration and delay functions      |

## §3.2 Expression Parsing: Shunting Yard Algorithm



### 3.2.1 Operator Precedence Table

| Operator                            | Precedence | Associativity |
|-------------------------------------|------------|---------------|
| Functions: $\sin$ , $\ln$ , $\dots$ | 5          | Right         |
| Power: $x^y$                        | 4          | Right         |
| $\times$ , $\div$ , $\text{mod}$    | 3          | Left          |
| $+$ , $-$                           | 2          | Left          |
| Parentheses                         | 1          | —             |

# Mathematical Engine — RK4 Method

## §4.1 ODE Formulation of Functions

| Function<br>$y(x)$ | ODE                       | Initial Conditions    |
|--------------------|---------------------------|-----------------------|
| $\sin(x)$          | $y'' + y = 0$             | $y(0) = 0, y'(0) = 1$ |
| $\sin^{-1}(x)$     | $\sqrt{1-x^2} y' - 1 = 0$ | $y(0) = 0$            |
| $\tan^{-1}(x)$     | $(1+x^2)y' - 1 = 0$       | $y(0) = 0$            |
| $\ln(x)$           | $x y' - 1 = 0$            | $y(1) = 0$            |
| $e^x$              | $y' - y = 0$              | $y(0) = 1$            |
| $x^w$              | $x y' - w y = 0$          | $y(1) = 1$            |

Derived relations:

$$\cos x = \sin\left(\frac{\pi}{2} - x\right), \quad \tan x = \frac{\sin x}{\cos x}, \quad \cos^{-1}(x) = \frac{\pi}{2} - \sin^{-1}(x), \quad \pi = 4 \tan^{-1}(1)$$

## §4.2 RK4 Update Equations

### RK4 — First-Order ODE

Given  $\frac{dy}{dx} = f(x, y)$ ,  $y(x_0) = y_0$ , step size  $h$ :

$$k_1 = h f(x_n, y_n) \quad (4.1)$$

$$k_2 = h f\left(x_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right) \quad (4.2)$$

$$k_3 = h f\left(x_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right) \quad (4.3)$$

$$k_4 = h f(x_n + h, y_n + k_3) \quad (4.4)$$

$$y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (4.5)$$

### 4.2.1 Second-Order Case: Sine Function

$y'' + y = 0$  is split into a first-order system  $\dot{y} = z$ ,  $\dot{z} = -y$ :

| Step   | $k_i$ for $\dot{y} = z$                                | $l_i$ for $\dot{z} = -y$                               |
|--------|--------------------------------------------------------|--------------------------------------------------------|
| 1      | $k_1 = h z_n$                                          | $l_1 = -h y_n$                                         |
| 2      | $k_2 = h(z_n + l_1/2)$                                 | $l_2 = -h(y_n + k_1/2)$                                |
| 3      | $k_3 = h(z_n + l_2/2)$                                 | $l_3 = -h(y_n + k_2/2)$                                |
| 4      | $k_4 = h(z_n + l_3)$                                   | $l_4 = -h(y_n + k_3)$                                  |
| Update | $y_{n+1} = y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ | $z_{n+1} = z_n + \frac{1}{6}(l_1 + 2l_2 + 2l_3 + l_4)$ |

## §4.3 Non-RK4 Functions

**Factorial:** Simple recursion  $y_n = n \cdot y_{n-1}$ ,  $y_0 = 1$ .

**Square Root:** Computed via the **Quake III** fast inverse square root, which exploits IEEE 754 bit representation:

**Listing 4.1.** Fast inverse square root (Quake III method)

```

1 float fast_inv_sqrt(float x) {
2     long i;
3     float x2 = x * 0.5f, y = x;
4     i = *(long *)&y;
5     i = 0x5f3759df - (i >> 1); // bit-level Newton seed
6     y = *(float *)&i;
7     y = y * (1.5f - (x2 * y * y)); // one Newton-Raphson step
8     return y;
9 }
10 // Usage: sqrt(x) = x * fast_inv_sqrt(x)

```

# Numerical Methods Comparative Analysis

---

## §5.1 Methods Evaluated

---

1. **Forward Euler:**  $y_{n+1} = y_n + h \cdot f(x_n, y_n)$
2. **Backward Euler:**  $y_{n+1} = y_n + h \cdot f(x_{n+1}, y_{n+1})$
3. **Trapezoidal:**  $y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$
4. **RK2 (Heun):** predictor–corrector with 2 function evaluations per step
5. **RK4:** 4 evaluations per step,  $O(h^4)$  local truncation error

## §5.2 Hardware Performance on ATmega328P

---

| Algorithm             | Flash (B)   | SRAM (B)  | Exec. Time ( $\mu$ s) |
|-----------------------|-------------|-----------|-----------------------|
| <b>RK4 (selected)</b> | <b>1392</b> | <b>16</b> | <b>192.31</b>         |
| Trapezoidal           | 1472        | 16        | 101.19                |
| Forward Euler         | 1272        | 16        | 45.75                 |
| Backward Euler        | 1296        | 16        | 55.56                 |

## §5.3 Accuracy Comparison for $\sin(x)$ , $h \approx 0.06$

---

Accuracy is measured in **Units in the Last Place (ULP)** vs. the MPFR multi-precision library:

$$\text{Error}_{\text{ULP}} = \frac{|y_{\text{calc}} - y_{\text{true}}|}{\varepsilon(y_{\text{true}})}$$

| Algorithm      | Max Error (ULP)                         | RMS Error                               | Stability   |
|----------------|-----------------------------------------|-----------------------------------------|-------------|
| <b>RK4</b>     | <b><math>2.38 \times 10^{11}</math></b> | <b><math>3.44 \times 10^{-7}</math></b> | <b>High</b> |
| Trapezoidal    | $6.00 \times 10^{14}$                   | $8.77 \times 10^{-4}$                   | High        |
| Heun (RK2)     | $1.20 \times 10^{15}$                   | $1.74 \times 10^{-3}$                   | Moderate    |
| Forward Euler  | $1.86 \times 10^{15}$                   | $8.56 \times 10^{-2}$                   | Unstable    |
| Backward Euler | $2.84 \times 10^{15}$                   | $7.27 \times 10^{-2}$                   | Unstable    |

#### Precision Floor Insight

On the ATmega328P, `avr-gcc` treats `float` and `double` as 32-bit IEEE 754. For RK4 the error reaches a **minimum at**  $h \approx 0.03$ . Smaller  $h$  only increases execution time because accumulated rounding error dominates — making any method of order  $> 4$  pointless on this platform. RK4 is the optimal choice.

#### Why RK4 Was Selected

RK4 achieves the **lowest error** at the hardware precision floor, guarantees **predictable fixed execution time** (no adaptive stepping), and the entire firmware fits in **12.3 KB Flash** and **350 B SRAM** — comfortable for the ATmega328P.

# Bug Report and Fix

## §6.1 Problem Statement

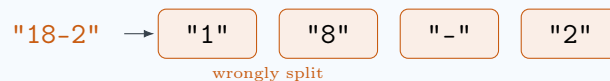
Entering the expression `18 - 2 =` caused the LCD to display `18-2=` on line 1 and show *nothing* on line 2 — the result `16` was never printed.

## §6.2 Root Cause Analysis

### Root Cause: Tokeniser in `pars.c`

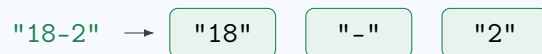
The tokenisation loop read one character at a time. When it encountered `'1'` followed by `'8'`, it treated them as two *separate* single-digit tokens instead of the number `18`. The resulting malformed postfix queue caused the RPN evaluator to produce no valid result.

**Before fix** — incorrect token stream:



Evaluator fails: `"1"` and `"8"` are not `18`

**After fix** — correct token stream:



Postfix: `18 2 -` ⇒ evaluates to `16`

## §6.3 Modules Modified and Fixes Applied

### 6.3.1 `pars.c` — Tokeniser Fix

**Listing 6.1.** Fixed digit-accumulation loop in `pars.c`

```
1  /* Accumulate ALL consecutive digit/decimal chars as one number
   token */
2  while (isdigit((unsigned char)*p) || *p == ',') {
```



```
3     token_buf[tok_len++] = *p++;
4 }
5 token_buf[tok_len] = '\0';
6 push_operand(atof(token_buf));    /* push complete multi-digit
    number */
```

### 6.3.2 keypad.c — Debounce and Index Fix

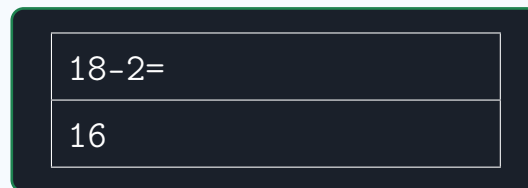
- Added 20 ms software debounce after a LOW column is detected
- Corrected the *(row,col)*-to-character lookup table index offsets

### 6.3.3 main.c — Buffer and Display Fix

- Expression buffer is now null-terminated before passing to parser
- = key handler clears LCD line 2 before printing, uses `dtostrf()` for float formatting

## §6.4 Verified Output After Fix

---



# Build and Upload Process

---

## §7.1 Toolchain

---

| Tool         | Purpose                                     |
|--------------|---------------------------------------------|
| avr-gcc      | Cross-compiler targeting ATmega328P         |
| avr-objcopy  | Converts .elf to .hex for upload            |
| platformio   | Build system managing toolchain and upload  |
| ArduinoDroid | Upload HEX to Arduino UNO via Android phone |

## §7.2 Software Prototype (Android / Linux)

---

**Listing 7.1.** Compile shared library and run Android server

```
# Clone the repository
git clone https://github.com/samikshas1118/Calculator

# Compile the C calculator library as a shared object
gcc -shared -o libcalc.so -fPIC func.c pars.c

# Run the Python server inside termux-debian (proot)
python3 server.py

# Install calci.apk on phone and open the app
```

## §7.3 Hardware Upload via PlatformIO

---

**Listing 7.2.** PlatformIO build and upload to Arduino UNO

```
# Create project
pio project init --board uno

# Copy all .c and .h source files to src/
# Then add to platformio.ini:
# build_flags = -Wl,-u,vfprintf -lprintf_flt
```

```
# Compile and upload  
pio run -t upload
```

#### Linker Flag Note

The flag `-lprintf_flt` enables floating-point formatting in `printf/sprintf` on AVR. Without it, `%f` format specifiers print nothing.

# Results and Conclusions

## §8.1 Final System Specifications

| Parameter        | Value                                              |
|------------------|----------------------------------------------------|
| Platform         | Arduino UNO (ATmega328P @ 16 MHz)                  |
| Flash Used       | 12.3 KB of 32 KB (38%)                             |
| SRAM Used        | 350 B of 2048 B (17%)                              |
| Numerical Method | RK4, fixed step $h = 0.03$                         |
| RMS Error (sin)  | $3.44 \times 10^{-7}$                              |
| Execution Time   | 192.3 $\mu$ s per function call                    |
| Input            | 25-key 5×5 matrix keypad                           |
| Output           | 16×2 LCD display                                   |
| Math Functions   | 15+ (trig, inv. trig, exp, log, power, sqrt, fact) |

## §8.2 Key Achievements

### Deliverables Completed

- ✓ 25-key matrix keypad with reliable debouncing
- ✓ 16×2 LCD display driver
- ✓ Complete RK4 software math engine for all transcendental functions
- ✓ Modular embedded C architecture (8 source files)
- ✓ Shunting Yard parser with precedence, associativity, and parentheses
- ✓ Multi-digit tokenisation bug identified and fixed
- ✓ Comparative analysis of 5 numerical methods with hardware benchmarks
- ✓ Firmware within tight ATmega328P resource constraints

### §8.3 Hardware Limitations

---

- No hardware FPU: `float` division costs 505 cycles vs.  $\approx 200$  for integer
- 32-bit software float imposes a hard precision floor ( $\approx 3 \times 10^{-7}$  RMS)
- Fixed-step RK4 trades latency ( $192 \mu s$ ) for maximum achievable accuracy

### §8.4 Possible Future Extensions

---

1. Migrate to STM32 with hardware FPU for double-precision results
2. Add complex number support
3. Implement a graphing mode with ST7735 graphical LCD
4. Port the RK4 engine to FPGA for parallel function evaluation
5. Add unit conversion and physical constants bank